

Coding Standards

Version: 1.0

Purpose

This document defines the coding standards for Trading Platform Pro.

The objective is to ensure a consistent, maintainable and high-quality codebase.

General Principles

Every line of code should be:

- readable
- maintainable
- testable
- deterministic

Code is written for humans first.

Python Version

Mandatory:

- Python 3.13

File Encoding

- UTF-8
- Unix line endings (LF)

Required Imports

Every Python module should begin with:

```
```python
from __future__ import annotations
```
```

Type Hints

Mandatory for:

- function parameters
- return values
- public attributes

Prefer explicit types over `Any`.

Naming

Modules

snake_case

Example:

```
```
```

market\_data\_service.py

```
```
```

Classes

PascalCase

Example:

MarketDataService

Functions

snake_case

Example:

calculate_position_size()

Variables

snake_case

Example:

market_price

Constants

UPPER_CASE

Example:

DEFAULT_TIMEOUT

Functions

Functions should:

- have one responsibility
- remain short
- be easy to test
- avoid side effects

Classes

Classes should:

- represent one concept
- have a single responsibility
- expose a clear API
- hide implementation details

Imports

Order:

1. Standard Library
2. Third-party Libraries

3. Project Imports
Avoid wildcard imports.

Error Handling

Errors should:

- fail explicitly
- provide meaningful messages
- never be silently ignored

Logging

Use the project logging infrastructure.

Never use:

```
```python  
print(...)
```
```

for production code.

Comments

Prefer self-explanatory code.

Comments should explain:

- why
- not
- what

Formatting

Formatting is enforced using:

- Ruff
- Ruff Format

Do not manually fight the formatter.

Testing

Every new feature should include appropriate tests.

Bug fixes should include regression tests whenever practical.

Code Review Checklist

Verify:

- readability
- naming
- typing
- architecture
- tests
- documentation

Async Programming

Use ``async`/`await`` consistently.

Rules:

- never block the event loop

- avoid nested event loops
 - use timeouts for external operations
 - support graceful cancellation
 - keep async functions focused
-

Dependency Injection

Dependencies shall be injected rather than created inside business logic.

Avoid:

- global service instances
 - hidden dependencies
 - service locators inside domain logic
-

Domain-Driven Design

Business rules belong to the Domain layer.

Application coordinates use cases.

Infrastructure provides technical implementations.

Presentation contains no business logic.

Clean Architecture

Always respect dependency direction.

Domain must remain independent from:

- infrastructure
 - UI
 - frameworks
 - external services
-

Configuration Management

Configuration shall:

- be externalized
 - support multiple environments
 - never contain secrets
 - remain backward compatible
-

Logging Standards

Use structured logging.

Every log entry should provide enough context for troubleshooting.

Never log:

- passwords
 - tokens
 - API keys
 - personal information
-

Security Standards

Never commit:

- credentials
- secrets
- certificates
- private keys

Use environment variables or secret providers.

Documentation Standards

Public modules, classes and functions should contain meaningful docstrings.
Documentation must stay synchronized with implementation.

Performance Guidelines

Optimize only after measurement.

Prefer:

- simple algorithms
- readable code
- predictable performance

Avoid premature optimization.

Code Smells

Avoid:

- duplicated code
- long methods
- deep nesting
- hidden side effects
- large classes
- magic numbers
- unused code

Related Documents

- Development_Guidelines.md
- Testing_Strategy.md
- AGENTS.md
- Architecture.md